PYTHON MID-SEMESTER: COMPLETE THEORY NOTES

B.Tech — 1st Semester | Python Programming

Purpose: These expanded notes explain the theory topics represented in the available mid-semester papers and mock papers. They include definitions, differences, examples, common mistakes, and practice questions so students can prepare for questions phrased in different ways.

Topics covered

- Variables, naming rules, data types, type(), and type casting
- Arithmetic, assignment, comparison, and logical operators
- Operator precedence, Boolean values, truthiness, and short-circuit evaluation
- Strings, len(), indexing, slicing, split(), and common string facts
- ASCII, Unicode, ord(), and chr()
- Input/output, range(), loops, and for vs while
- break, continue, pass, nested conditions, nested loops, and pattern logic
- if-elif-else, match-case, syntax errors, runtime errors, and logical errors
- Definitions, differences, explanations, output tracing, and revision questions

1. Variables and Variable Names

1.1 What is a variable?

A variable is a name used to store or refer to a value in a program. You can use the variable name later instead of writing the value again.

```
student_name = "Riya"
age = 18
print(student_name)
print(age)
```

Here, `student_name` refers to the string value `"Riya"` and `age` refers to the integer value `18`.

1.2 Rules for valid variable names

- A name can contain letters (A–Z, a–z), digits (0–9), and underscores (`_`).
- A name cannot begin with a digit.
- Spaces are not allowed in a variable name.
- Symbols such as `@`, `#`, `-`, and `!` are not allowed in ordinary variable names.
- Python keywords such as `if`, `else`, `for`, `while`, and `class` cannot be used as variable names.
- Python variable names are case-sensitive: `age`, `Age`, and `AGE` are different names.

Examples:

- Valid: `total_marks`, `student2`, `_count`, `user_age`
- Invalid: `total marks` (space), `2student` (starts with a digit), `first-name` (hyphen), `class` (keyword)

1.3 Assignment statement

The equal sign (=) assigns a value to a variable. It does not mean “is equal to” in the comparison sense.

```
marks = 85
city = "Delhi"
price = 99.5
```

Remember: Use meaningful names such as `total_marks` or `student_age`. They make code easier to read.

Check your understanding

1. 1. Define a variable and write one example.
2. 2. Write three rules for naming variables.
3. 3. Are `marks` and `Marks` the same variable name in Python? Explain.
4. 4. Identify valid names: `name_1`, `1name`, `first name`, `_total`, `while`.

2. Data Types, type(), and Type Casting

2.1 What is a data type?

A data type tells Python what kind of value is being used. Different types support different kinds of operations.

- `int`: whole numbers, such as `12`, `-5`, and `0`
- `float`: numbers with a decimal part, such as `2.5` and `-0.75`
- `str`: text or a sequence of characters, such as `"Python"` and `"123"`
- `bool`: truth values, `True` and `False`

2.2 The type() function

The `type()` function reports the type of a value or variable.

```
print(type(12))       # int
print(type(4.25))     # float
print(type("12"))     # str
print(type(False))    # bool
```

Common confusion: `12` is an integer, but `"12"` is a string because it is inside quotes.

2.3 What is type casting?

Type casting means converting a value from one data type to another. Common conversion functions are `int()`, `float()`, and `str()`.

```
a = int("25")       # 25
b = float("3.5")    # 3.5
c = str(100)        # "100"
d = int(4.9)        # 4
```

`int(4.9)` gives `4` because conversion to `int` removes the decimal part; it does not round to the nearest integer.

2.4 Why do we need type casting?

The `input()` function returns text. If a user enters 20 and the program needs to perform numerical calculations, convert the input to `int` or `float` first.

```
age_text = input("Enter age: ")
age = int(age_text)
print(age + 1)
```

Without `int()`, `age_text` is a string, so adding the integer `1` to it would cause a type error.

Check your understanding

5. 1. What information does `type()` provide?
6. 2. Differentiate between `25` and `"25"`.
7. 3. Define type casting and name three conversion functions.
8. 4. What is the result of `int(8.7)`? Why?
9. 5. Why is type casting often needed after `input()`?

3. Operators in Python

An **operator** is a symbol or keyword that performs an operation on values. The values on which an operator works are called **operands**.

3.1 Arithmetic operators

- `+` addition: `8 + 3` gives `11`
- `-` subtraction: `8 - 3` gives `5`
- `*` multiplication: `8 * 3` gives `24`
- `/` division: `8 / 3` gives approximately `2.6667`
- `//` floor division: `8 // 3` gives `2`
- `%` modulo/remainder: `8 % 3` gives `2`
- `**` exponentiation/power: `2 ** 4` gives `16`

3.2 Important differences: /, //, and %

The `/` operator performs ordinary division. The `//` operator performs floor division. The `%` operator gives the remainder after division.

```
print(17 / 5)    # 3.4
print(17 // 5)   # 3
print(17 % 5)    # 2
```

These operators are useful in number-based problems. For a positive integer `n`, `n % 10` gives its last digit, while `n // 10` removes its last digit.

```
n = 583
print(n % 10)    # 3
print(n // 10)   # 58
```

3.3 Assignment and augmented assignment

The assignment operator `=` stores a value in a variable. Augmented assignment updates an existing variable using an operation.

- `x += 3` means `x = x + 3`
- `x -= 2` means `x = x - 2`
- `x *= 4` means `x = x * 4`
- `x /= 2` means `x = x / 2`
- `x //= 3` means `x = x // 3`
- `x %= 3` means `x = x % 3`
- `x **= 2` means `x = x ** 2`

Example:

```
x = 10
x += 3
print(x)   # 13
x *= 2
print(x)   # 26
```

3.4 Comparison operators

Comparison operators compare two values and produce a Boolean result: `True` or `False`.

- `==` equal to: `5 == 5` gives `True`
- `!=` not equal to: `5 != 3` gives `True`
- `>` greater than: `5 > 3` gives `True`
- `<` less than: `5 < 3` gives `False`
- `>=` greater than or equal to: `5 >= 5` gives `True`
- `<=` less than or equal to: `4 <= 2` gives `False`

Do not confuse `=` with `==`. The first assigns a value; the second checks whether two values are equal.

3.5 Logical operators

Logical operators combine or modify conditions.

- `and`: the combined expression is true when both operands are truthy.
- `or`: the combined expression is true when at least one operand is truthy.
- `not`: reverses a truth value.

```
print(True and False)   # False
print(True or False)    # True
print(not True)         # False
```

3.6 Truthy and falsy values

In a Boolean context, Python treats some values as false and most other values as true. Common falsy values include `False`, `0`, `0.0`, and an empty string `""`. A non-zero number and a non-empty string are truthy.

```
bool(0)       # False
bool(12)      # True
bool("")      # False
bool("Hi")    # True
bool(-3)      # True
```

3.7 How and/or expressions work

Python's `and` and `or` operators do not always return the Boolean words `True` or `False`. They may return one of their operands.

- `A or B`: if `A` is truthy, Python returns `A` without needing to evaluate `B`. Otherwise, it evaluates and returns `B`.
- `A and B`: if `A` is falsy, Python returns `A` without needing to evaluate `B`. Otherwise, it evaluates and returns `B`.

```
print(0 or 7)           # 7
print(5 or 7)           # 5
print(0 and 7)          # 0
print(5 and 7)          # 7
print(0 or 7 + 6 or 0)  # 13
```

In the last expression, addition is evaluated before `or`. So `7 + 6` becomes `13`, then the `or` operations are evaluated from left to right.

3.8 Operator precedence

Operator precedence determines which operation is performed first when an expression contains several operators. Parentheses can change the order.

A useful order to remember for these topics:

10. 1. Parentheses
11. 2. Exponentiation (`**`)
12. 3. Unary plus/minus
13. 4. Multiplication, division, floor division, modulo (`*`, `/`, `//`, `%`)
14. 5. Addition and subtraction (`+`, `-`)
15. 6. Comparisons (`==`, `!=`, `<`, `>`, `<=`, `>=`)
16. 7. `not`
17. 8. `and`
18. 9. `or`

```
print(5 + 3 * 2 ** 2)  # 17
# 2**2 = 4; 3*4 = 12; 5+12 = 17
```

Exam strategy: For a tricky expression, calculate arithmetic first, then comparisons, then `not`/`and`/`or` as applicable.

Check your understanding

19. 1. Explain any four arithmetic operators.
20. 2. Differentiate between `/`, `//`, and `%` with examples.

21. 3. What is the difference between `=` and `==`?
22. 4. What is an augmented assignment operator? Give two examples.
23. 5. Name the six common comparison operators.
24. 6. Explain `and`, `or`, and `not`.
25. 7. What are truthy and falsy values? Give two examples of each.
26. 8. Explain why an `and`/`or` expression can return a number rather than `True` or `False`.
27. 9. What is operator precedence? Evaluate `5 + 3 * 2 ** 2`.

4. Strings and Common String Operations

4.1 What is a string?

A string is a sequence of characters written inside single or double quotes.

```
first = "Python"
second = 'Programming'
number_text = "123"
```

The value `"123"` is a string, not an integer, because it is enclosed in quotes.

4.2 len() function

`len()` returns the number of characters in a string, including spaces and punctuation.

```
print(len("Python"))      # 6
print(len("Hi there"))    # 8
print(len("A!"))          # 2
```

4.3 String indexing

Indexing accesses one character by position. Positive indexes begin at `0` from the left. Negative indexes begin at `-1` from the right.

For `"Python"`:

- Positive indexes: P=0, y=1, t=2, h=3, o=4, n=5
- Negative indexes: P=-6, y=-5, t=-4, h=-3, o=-2, n=-1

```
word = "Python"
print(word[0])   # P
print(word[2])   # t
print(word[-1])  # n
```

4.4 String slicing

Slicing extracts a part of a string. Its general form is `text[start:stop:step]`. The start position is included, but the stop position is excluded.

Slice • Result • Explanation

--- • --- • ---

`"Python"[1:4]` • `"yth"` • Indexes 1, 2, 3

`"Python"[:3]` • `"Pyt"` • Starts at beginning

`"Python"[2:]` • `"thon"` • Continues to end

`"Python"[-3:]` • `"hon"` • Last three characters

`"Python"[::2]` • `"Pto"` • Every second character

`"Python"[::-1]` • `"nohtyP"` • Reads backwards

`"Python"[4:1:-1]` • `"oht"` • Moves backward; stop excluded

If a slice's stop value is outside the string length, Python returns the available characters rather than raising an index error. If the start/stop/step combination selects no positions, the result is an empty string.

4.5 split() method

The `split()` method divides a string into parts and returns a list of strings. Without a separator, it splits on whitespace. With a separator, it splits at that separator.

```
"red blue green".split()     # ['red', 'blue', 'green']
"a,b,c".split(",")           # ['a', 'b', 'c']
"one-two-three".split("-")   # ['one', 'two', 'three']
```

For theory answers, state what `split()` does, mention the separator, and show a small example.

4.6 Useful string facts

- Strings are written inside quotes.
- Spaces are characters and are counted by `len()`.
- String indexes start at `0`.
- Slicing includes the start but excludes the stop.
- Negative indexes count from the end.
- The step controls how the slice moves through the string.
- `split()` returns a list of strings; it does not return one combined string.

Check your understanding

28. 1. Define a string and give an example.
29. 2. What does `len()` return for a string?
30. 3. Explain positive and negative string indexes.
31. 4. What is slicing? Explain start, stop, and step.
32. 5. Why is the stop index excluded in a slice?
33. 6. What does `split()` do? Give two examples.
34. 7. Predict: `"computer"[1:4]`, `"python"[-2:]`, and `"python"[::-1]`.

5. ASCII, Unicode, ord(), and chr()

5.1 Character encoding

Computers store information as numbers. Character encoding provides a mapping between characters and numeric codes.

5.2 ASCII

ASCII is an older character-encoding standard that represents English letters, digits, punctuation, and control characters using numeric codes. Standard ASCII uses 7 bits and defines 128 code values.

5.3 Unicode

Unicode is a broader standard that assigns a code point to characters from many languages and symbol sets. Python strings support Unicode characters. The original ASCII characters have the same code points in Unicode; for example, `'A'` is 65.

5.4 ord() and chr()

- `ord(character)` returns the integer code point for a single character.
- `chr(number)` returns the character associated with a valid Unicode code point.

```
print(ord('A'))  # 65
print(ord('a'))  # 97
print(chr(65))   # A
print(chr(97))   # a
```

Remember: `ord()` goes from character to number; `chr()` goes from number to character.

Check your understanding

35. 1. What is character encoding?

36. 2. What is ASCII?
37. 3. How is Unicode broader than ASCII?
38. 4. What does `ord()` return? Give an example.
39. 5. What does `chr()` return? Give an example.
40. 6. Find `ord('B')` and `chr(67)`.

6. Input, Output, and Formatting

6.1 print() function

`print()` displays values or text on the screen. It can display multiple values separated by commas.

```
print("Hello")
name = "Aman"
print("Welcome", name)
print("Sum =", 4 + 5)
```

6.2 input() function

`input()` displays an optional prompt, waits for the user to type something, and returns the entered value as a string.

```
name = input("Enter name: ")
print("Hello", name)
```

6.3 Converting user input

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
print("Sum =", num1 + num2)
```

If the input is not converted, the plus operator may join strings instead of adding numbers.

```
a = input()   # user types 2; a is "2"
b = input()   # user types 3; b is "3"
print(a + b)  # "23", not 5
```

6.4 Output formatting basics

Commas and f-strings can help display readable output.

```
name = "Riya"
marks = 92
print("Name:", name, "Marks:", marks)
print(f"{name} scored {marks} marks.")
```

Check your understanding

41. 1. What does `print()` do?
42. 2. What does `input()` return?
43. 3. Why might `int(input())` be needed in a numerical program?
44. 4. What is the difference between adding strings `"2"` and `"3"` and adding integers `2` and `3`?
45. 5. Write a short program that accepts a name and prints a greeting.

7. Loops and range()

7.1 What is a loop?

A loop repeats a block of code. It avoids writing the same statements many times and is useful for counting, processing digits, calculating totals, and printing patterns.

7.2 for loop

A `for` loop takes values from a sequence or iterable, one at a time, and executes the loop body for each value.

```
for i in range(1, 4):
    print(i)
# Output: 1, 2, 3
```

7.3 while loop

A `while` loop repeats while its condition is true. The loop body should usually update the values involved in the condition so the loop can eventually stop.

```
i = 1
while i <= 3:
    print(i)
    i += 1
```

Common mistake: If a `while` condition never becomes false and there is no `break`, the program may run indefinitely.

7.4 Difference between for and while

- `for` is often used when iterating through a sequence or known range.
- `while` is often used when repetition depends on a condition.
- In a `for` loop, the next value is supplied by the iterable or `range()`.
- In a `while` loop, the programmer commonly updates the control variable.

7.5 range() function

`range()` produces a sequence of integers. Its common forms are `range(stop)`, `range(start, stop)`, and `range(start, stop, step)`. The stop value is excluded.

Expression • Values generated

--- • ---

`range(5)` • 0, 1, 2, 3, 4

`range(2, 7)` • 2, 3, 4, 5, 6

`range(2, 10, 3)` • 2, 5, 8

`range(6, 0, -2)` • 6, 4, 2

`range(3, 3)` • No values

A positive step moves upward; a negative step moves downward. A step of zero is invalid.

7.6 Infinite loop

An infinite loop is a loop that does not terminate by itself. It may happen when a `while` condition remains true because the control variable is not updated correctly.

```
i = 1
while i <= 3:
    print(i)
    # Missing i += 1: condition stays true
```

Check your understanding

46. 1. Define a loop and state one advantage.
47. 2. Explain the difference between `for` and `while`.
48. 3. What does `range()` do?
49. 4. Why is the stop value of `range()` excluded?
50. 5. What values are generated by `range(2, 9, 2)` and `range(5, 0, -1)`?
51. 6. What is an infinite loop, and how can it occur?

8. break, continue, pass, and Loop else

8.1 break

`break` immediately terminates the nearest enclosing loop. No further iterations of that loop occur.

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
# Output: 1 2
```

8.2 continue

`continue` skips the rest of the current iteration and moves to the next iteration.

```
for i in range(1, 5):
    if i == 3:
        continue
    print(i)
# Output: 1 2 4
```

8.3 pass

`pass` is a placeholder statement. It performs no action but lets the code remain syntactically valid when a statement is required.

```
for i in range(3):
    pass  # Add code here later
```

8.4 break vs continue vs pass

- `break`: terminates the nearest loop.
- `continue`: skips the rest of the current iteration; the loop continues with the next iteration.
- `pass`: does nothing; execution moves to the next statement normally.

8.5 for-else / while-else (review if taught)

A loop can have an `else` block. The `else` block runs when the loop finishes normally, but it is skipped if the loop ends through `break`.

```
for i in range(3):
    print(i)
else:
    print("Loop completed")

for i in range(3):
    if i == 1:
        break
else:
    print("This does not print")
```

Check your understanding

52. 1. Explain `break` and `continue` with examples.
53. 2. What is the purpose of `pass`?
54. 3. Differentiate between `break`, `continue`, and `pass`.
55. 4. What happens to a loop when `break` executes?
56. 5. What happens to the current iteration when `continue` executes?
57. 6. When does a loop's `else` block execute?

9. Conditional Statements and Nested if

9.1 What is a conditional statement?

A conditional statement allows a program to choose which statements to execute based on whether a condition is true or false.

9.2 if statement

An `if` block executes only when its condition is true.

```
age = 20
if age >= 18:
    print("Adult")
```

9.3 if-else statement

`if-else` provides two paths: one for a true condition and another for a false condition.

```
number = 7
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

9.4 if-elif-else statement

`elif` means “else if.” Python checks conditions from top to bottom. The first true branch executes; remaining branches are skipped. `else` runs if none of the earlier conditions is true.

```
marks = 76
if marks >= 90:
    print("Excellent")
elif marks >= 70:
    print("Good")
elif marks >= 40:
    print("Pass")
else:
    print("Needs Improvement")
```

9.5 Nested if

A nested `if` is an `if` statement inside another `if` or `else` block. The inner condition is checked only if execution enters the outer block containing it.

```
age = 20
has_id = True
if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Too young")
```

Nested conditions are useful when a decision depends on more than one step, such as first checking age and then checking whether an ID is available.

9.6 match-case

`match-case` compares one value against several possible cases. It is useful when a value must be matched against fixed alternatives. The underscore case _ works as a default case.

```
day = "Sunday"
match day:
    case "Saturday" | "Sunday":
        print("Weekend")
    case "Monday" | "Tuesday" | "Wednesday" | "Thursday" | "Friday":
        print("Weekday")
    case _:
        print("Invalid day")
```

The | symbol in a pattern means either listed pattern can match. Python 3.10 or later is required for `match-case`.

9.7 if-elif-else vs match-case

- `if-elif-else` can test ranges and complex Boolean conditions, such as `marks >= 90`.
- `match-case` is most useful for matching a value against distinct patterns or cases, such as a day name.
- In `if-elif-else`, conditions are expressions. In `match-case`, cases are written under `match`.

Check your understanding

58. 1. What is the purpose of a conditional statement?
59. 2. Explain `if`, `if-else`, and `if-elif-else`.
60. 3. What is a nested `if` statement? Give a small example.
61. 4. When is a nested `if` useful?
62. 5. What is `match-case`? What does `case _` mean?
63. 6. Give one difference between `if-elif-else` and `match-case`.

10. Nested Loops and Pattern Printing

10.1 What is a nested loop?

A nested loop is a loop inside another loop. The outer loop controls the main repetitions; the inner loop repeats for each outer-loop iteration.

```
for i in range(1, 4):
    for j in range(1, 3):
        print(i, j)
```

The outer loop takes 1, 2, and 3. For each outer value, the inner loop takes 1 and 2. Therefore, the `print` statement runs 3 × 2 = 6 times.

10.2 Counting inner-loop executions

```
for i in range(3):       # 3 outer iterations
    for j in range(4):   # 4 inner iterations per outer iteration
        print(i, j)      # 3 * 4 = 12 executions
```

If the inner loop's number of iterations changes with `i`, calculate each row separately and add the totals.

```
for i in range(1, 4):
    for j in range(i):
        print("*", end="")
    print()
# *
# **
# ***
```

10.3 print() and end

By default, `print()` moves to a new line after printing. The `end` parameter changes what is printed at the end. Using `end=""` keeps the next output on the same line.

```
print("*")
print("*")
# Two lines

print("*", end="")
print("*", end="")
# Same line: **
```

10.4 Indentation and sibling loops

Indentation defines which loop owns a statement. Two loops at the same indentation level are sibling loops, not nested loops.

```
for i in range(2):
    print("A")
for j in range(2):
    print("B")
# First loop completes, then second loop runs.
```

10.5 Pattern printing: how to think

- The outer loop usually controls the row number.

- The inner loop controls how many characters or numbers are printed in each row.
- `print(..., end="")` keeps output on the same line.
- A plain `print()` after the inner loop moves to the next row.
- For a decreasing pattern, the inner-loop count often decreases as the row number increases.

```
n = 4
for i in range(1, n + 1):
    for j in range(i):
        print("*", end="")
    print()
# *
# **
# ***
# ****
```

Check your understanding

64. 1. Define a nested loop.
65. 2. How many times does the inner statement execute if the outer loop runs 4 times and the inner loop runs 3 times per outer iteration?
66. 3. What is the purpose of `end=""` in `print()`?
67. 4. How can you identify whether two loops are nested or siblings?
68. 5. Explain the roles of outer and inner loops in pattern printing.

11. Errors and Debugging Basics

11.1 Syntax error

A syntax error occurs when code breaks Python's grammar rules. Python cannot properly parse the code, so it reports an error before normal execution.

```
if age >= 18
    print("Adult")  # Missing colon
```

11.2 Logical error

A logical error occurs when a program runs but gives the wrong answer because the algorithm, formula, or condition is incorrect.

```
length = 5
width = 3
area = length + width  # Runs, but should use multiplication
```

11.3 Runtime error

A runtime error happens while a program is executing, for example dividing by zero or converting invalid text to an integer.

```
print(10 / 0)        # ZeroDivisionError
print(int("hello"))  # ValueError
```

Differences:

- **Syntax error:** code grammar is invalid, such as a missing colon.
- **Runtime error:** an error occurs during execution, such as `10 / 0`.
- **Logical error:** program runs but the result is wrong, such as using `+` instead of `*` for area.

11.4 Basic debugging approach

69. 1. Read the error message and locate the line mentioned.
70. 2. Check punctuation such as colons, quotes, brackets, and parentheses.
71. 3. Check indentation in `if` statements and loops.
72. 4. Print intermediate values to see how variables change.
73. 5. Test simple cases as well as boundary cases.

Check your understanding

74. 1. Differentiate between syntax and logical errors.
75. 2. Give one example of a runtime error.
76. 3. What is debugging?
77. 4. Name three things to check when a Python program gives an error.

12. How to Write Strong Theory Answers

For a 1-mark question, write a direct definition and, if requested, one example. For a 5-mark explain question, use this structure:

78. 1. **Definition:** explain the concept in one or two sentences.
79. 2. **Key points:** list the important rules or features.
80. 3. **Example:** write a short, correct Python example.
81. 4. **Explanation/output:** state what the example demonstrates.
82. 5. **Difference table:** when asked to differentiate two concepts, compare them point by point.

Example answer structure for “Explain break and continue”:

- `break` terminates the nearest loop immediately.
- `continue` skips the rest of the current iteration and moves to the next iteration.
- Add a short example for each and state its output.

13. Theory Practice Bank

Practise answering these without looking at the notes. They cover definitions, explanations, differences, and short output-tracing styles represented by the paper topics.

A. Definitions

83. 1. Define a variable and write one example.
84. 2. What does `type()` tell us about a value?
85. 3. Define type casting and name two functions used for it.
86. 4. What is a string? Give one example.
87. 5. What does `len()` return when used with a string?
88. 6. What is a loop? State one purpose of using loops.
89. 7. What is a nested conditional statement?
90. 8. What is a nested loop?
91. 9. What is string slicing?
10. What does `split()` do?
11. What are the two Boolean values in Python?
12. What is the purpose of `range()`?
13. What are ASCII and Unicode?
14. What do `ord()` and `chr()` return?
15. What is a syntax error?
16. What is a logical error?

B. Explain / Give Examples

92. 1. Explain arithmetic operators with at least four examples.
93. 2. Explain comparison operators with examples.
94. 3. Explain logical operators `and`, `or`, and `not`.
95. 4. Explain the difference between `/`, `//`, and `%`.
96. 5. Explain assignment and augmented assignment operators.
97. 6. Explain how operator precedence changes the result of an expression.
98. 7. Explain type casting with three examples.

8. Explain string slicing and `start`, `stop`, and `step`.
9. Explain the purpose of `input()` and `print()`.
10. Explain how `range(start, stop, step)` works.
11. Explain the purpose of `break`, `continue`, and `pass`.
12. Explain nested `if` statements with a simple example.
13. Explain nested loops with a simple example.
14. Explain `match-case` with a simple example.
15. Explain why indentation is important in loops and conditions.
16. Explain how to find and fix a syntax error.

C. Differentiate Between

1. `int` and `float`
2. Integer `123` and string `"123"`
3. `=` and `==`
4. `/` and `//`
5. `//` and `%`
6. `for` loop and `while` loop
7. `break` and `continue`
8. `continue` and `pass`
9. Syntax error and logical error
10. ASCII and Unicode
11. `if-elif-else` and `match-case`
12. Positive indexing and negative indexing
13. String slicing and `split()`

D. Short Application / Trace Questions

1. What is the output of `print(17 // 5)` and `print(17 % 5)`?
2. What is the output of `print(type("25"))`?
3. What is the output of `print("Python"[1:4])`?
4. What is the output of `print("Python"[::-1])`?
5. What does `range(2, 8, 2)` generate?
6. What does `range(5, 0, -2)` generate?
7. What is the output of `print(0 or 9)`?
8. What is the output of `print(0 and 9)`?
9. What is the output of `print(0 or 7 + 6 or 0)`?
10. How many times does the inner statement execute in a 3-by-4 nested loop?
11. What does `continue` do when `i == 3` in a loop from 1 to 4?
12. What is wrong with `if x > 5 print(x)`?

14. Final Revision Checklist

- I can define variables, data types, strings, loops, nested `if`, and nested loops.
- I can explain `type()`, `len()`, `split()`, `bool()`, `range()`, `ord()`, and `chr()`.
- I know the differences between `=` and `==`, `/` and `//`, and `//` and `%`.
- I can explain arithmetic, assignment, comparison, and logical operators.
- I can evaluate expressions using operator precedence and `and`/`or`/`not`.
- I can explain string indexing and slicing, including negative steps.
- I can explain `input()` and why its result may need type casting.
- I can explain `for` vs `while` and `range()` start/stop/step.
- I can explain `break`, `continue`, `pass`, and loop `else` if it was taught.

- I can explain `if-elif-else`, nested `if`, and `match-case`.
- I can explain nested loops, inner-loop counts, and `print(end="")`.
- I can distinguish syntax, runtime, and logical errors.
- I can answer the practice questions in my own words and write short examples.

End of Theory Notes

These notes are prepared from theory-oriented portions and concepts represented in the available Python mid-semester papers and mock papers. Exact examination questions cannot be predicted.